



ChatGPT와 함께하는 딥러닝 5일 완성

전이 학습 및 이미지 분류 실습

4일차



서울대학교 평생교육원
Extension College Seoul National University

Topics

- 전이 학습 개념 및 활용 목적 이해
- 사전 학습된 대표 모델(VGG16, ResNet50 등)의 구조
- 전이 학습 기반 이미지 분류 모델 구축
- 일부 레이어 미세 조정 및 재학습
- 허깅페이스 모델 활용 이미지 분류 실습

이미지 분류 작업순서

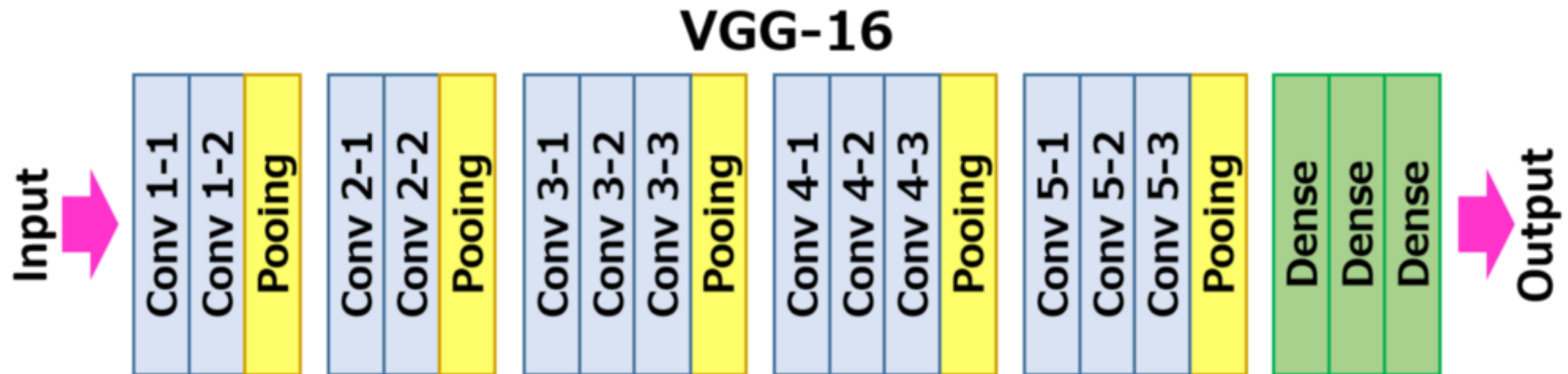


VGG-16

224x224 pixels colored
images 사용

Error - 7.3
19 layers

- A simpler architecture model with less hyper parameters. 하이퍼 파라미터가 적은 단순한 아키텍처 모델
- Trained using more than 1 million images from the ImageNet database and can classify up to 1000 objects. 백만장 이상 이미지를 훈련시켜서 1000개 사물을 분류



Save and Load Model 모델저장 및 로딩

모델 저장할 때

```
model.save('cifar10.keras')
```

전체 모델 → .keras
가중치만 → .weights.h5

나중에 사용할 때 (컴파일 할 필요 없음)

```
from keras.models import load_model
```

```
new_model = load_model('fashion_mnist.keras')
```

```
pred = np.argmax(new_model.predict(X_test), axis=1)
```

```
pred[0]
```

Save and Load Model 모델저장 및 로딩

가중치만 저장

```
model.save_weights("cifar10_weights.weights.h5")
```

가중치 불러오기

```
model.load_weights("cifar10_weights.weights.h5")
```

테스트 데이터로 예측

테스트 데이터 열기

```
from PIL import Image
```

```
airplane =  
Image.open('/content/airplane.jp  
g')  
airplane
```



전처리

```
# 사이즈 변경 (32,32)
```

```
airplane =  
airplane.resize((32,32))
```

```
airplane
```

```
import numpy as np
```

```
airplane = np.array(airplane) #  
넘파이 배열로 변환
```

```
airplane =  
airplane.reshape(1,32,32,3) # 4차  
원으로 변환
```

```
airplane = airplane / 255 # 정규  
화
```


예측

예측 후 이름 매칭

```
label_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',  
'dog', 'frog', 'horse', 'ship', 'truck']
```

```
label_names[np.argmax(model.predict(airplane))]
```

1/1  0s 60ms/step

'airplane'

CNN 모델 성능향상 기법

- Dropout Regularization (20%, 30%, 50%) 드롭아웃
- Weight Regularization 가중치 정규화
- Data Augmentation 데이터 증강
- Dropout and Data Augmentation 드롭아웃 + 데이터증강
- Dropout and Data Augmentation and Batch Normalization 드롭아웃 + 데이터증강 + 배치정규화

Image Data Augmentation

이미지 데이터 증강

- A technique used to artificially increase the diversity of your training dataset by applying random transformations to the existing images. 기존 이미지에 무작위 변환을 적용하여 훈련 데이터 세트의 다양성을 인위적으로 늘리는 데 사용되는 기술
- Helps improve the generalization and robustness of deep learning models, especially in tasks like image classification, object detection, and segmentation. 딥러닝 모델의 일반화와 견고성을 개선하는 데 도움이 되며, 특히 이미지 분류, 객체 감지 및 세분화와 같은 작업에서 도움이 됨
- Increases data variety, reduces the need for data, and produces the realistic performance. 데이터 다양성을 높이고, 데이터의 필요성을 줄이며, 사실적인 성능을 생성

Image Data Augmentation Techniques 이미지 증강 기법

- Flipping
 - Horizontal Flip: 이미지를 수평으로 뒤집어 거울 이미지를 만듦
 - Vertical Flip: 이미지를 수직으로 뒤집음
- Rotation: 지정된 각도로 이미지를 회전하여 개체 방향에 변형을 적용
- Zooming: 이미지를 확대하거나 축소하여 이미지에 있는 개체의 배율을 변경
- Shifting
 - Width Shift: 이미지를 수평으로 이동
 - Height Shift: 이미지를 수직으로 이동
- Shearing: 이미지의 한 부분을 지정된 각도로 이동하여 이미지를 기울임
- Brightness Adjustment: 이미지의 밝기를 임의로 조정
- Contrast Adjustment: 이미지의 대비를 임의로 조정
- Noise Injection: 이미지에 무작위 노이즈를 추가

Image Data Augmentation Code

이미지 데이터 증강

```
from keras.preprocessing.image import ImageDataGenerator
```

```
train_datagen = ImageDataGenerator(  
    width_shift_range=0.1,  
    height_shift_range=0.1,  
    horizontal_flip=True  
)
```

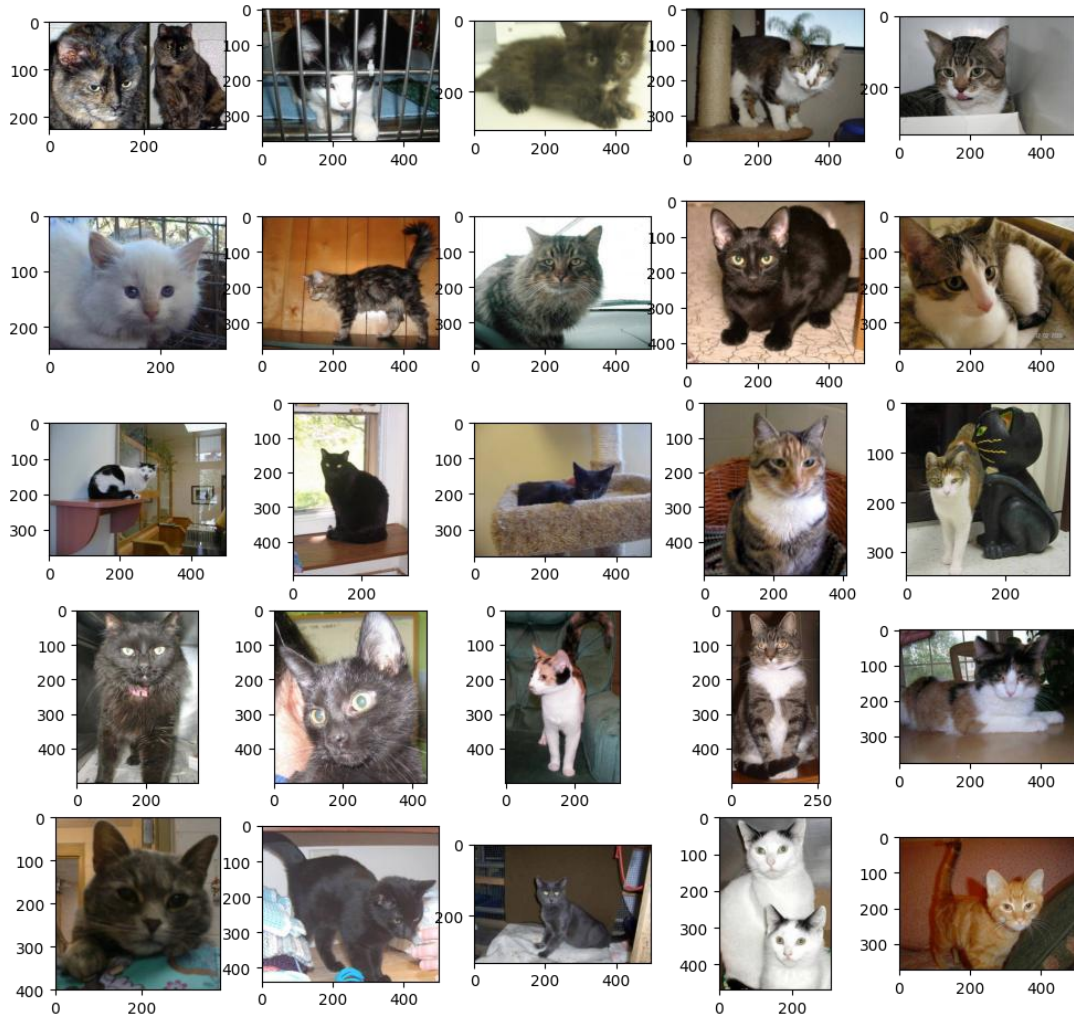
```
train_flow = train_datagen.flow(x_train, y_train, batch_size=64)
```

```
test_datagen = ImageDataGenerator()  
test_flow = test_datagen.flow(x_test, y_test, batch_size=64)
```

모델 학습

```
history = model6.fit(train_flow, epochs=100, batch_size=64,  
    validation_data=test_flow, verbose=1, callbacks=[checkpoint,  
    earlystopping])
```

Cats and Dogs Dataset



- 마이크로 소프트 사이트

- <https://www.microsoft.com/en-us/download/details.aspx?id=54765>

- 캐글

- <https://www.kaggle.com/competitions/dogs-vs-cats/data>

- 구글

- https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip

Lab #4

- 캐글에서 cats_and_dogs 파일 다운로드 받고 압축풀기
- Train and validation directory에 있는 이미지 데이터 증강 생성 (150x150)
- CNN 모델 생성 및 훈련, 히스토리 그래프
- 테스트 이미지 분류

Dogs Vs. Cats 데이터 다운로드

다운로드 (리눅스)

```
!wget --no-check-certificate \ #SSL 인증서 검사를 비활성화
      https://storage.googleapis.com/mledu-
datasets/cats_and_dogs_filtered.zip \
      -O /tmp/cats_and_dogs_filtered.zip #저장이름
```

다운로드 (윈도우)

```
!curl -o /tmp/cats_and_dogs_filtered.zip -k
https://storage.googleapis.com/mledu-
datasets/cats_and_dogs_filtered.zip
```


Dogs Vs. Cats 데이터 다운로드

다운로드 (캐글)

```
!pip install Kaggle
```

```
!kaggle datasets download -d tongpython/cat-and-dog
```

압축 풀어서 저장

```
!unzip cat-and-dog.zip -d /content/cats_and_dogs
```

폴더에 압축 푸는 방법

압축풀기

```
import os
```

```
import zipfile
```

```
local_zip = '/content/cats_and_dogs.zip'
```

```
zip_ref = zipfile.ZipFile(local_zip, 'r')
```

```
zip_ref.extractall('/content/cats_and_dogs')
```

```
zip_ref.close()
```

폴더에 있는 파일 이름 출력

파일 이름 출력

```
import os
os.listdir('cats_and_dogs/training_set/trainin
g_set/cats')
```

이미지 개수

```
len(os.listdir('cats_and_dogs/training_set/tra
ining_set/cats'))
```

```
['cat.2052.jpg',
 'cat.1346.jpg',
 'cat.35.jpg',
 'cat.3948.jpg',
 'cat.2014.jpg',
 'cat.632.jpg',
 'cat.582.jpg',
 'cat.3459.jpg',
 'cat.3990.jpg',
 'cat.11.jpg',
 'cat.378.jpg',
 'cat.3364.jpg',
 'cat.1975.jpg',
 'cat.2908.jpg',
 'cat.2938.jpg',
 'cat.1915.jpg',
```

훈련 및 검증 디렉토리 설정

- os.path.join을 이용해도 됨

패쓰설정

```
train_dir = '/content/cats_and_dogs/training_set/training_set'
```

```
test_dir = '/content/cats_and_dogs/test_set/test_set'
```

훈련 고양이, 강아지 디렉토리 설정

패스 이름 만들기

```
train_cat_dir =  
'/content/cats_and_dogs/training_set/training_set/cats'
```

```
train_dog_dir =  
'/content/cats_and_dogs/training_set/training_set/dogs'
```

```
val_cat_dir = '/content/cats_and_dogs_filtered/validation/cats'
```

```
val_dog_dir = '/content/cats_and_dogs_filtered/validation/dogs'
```

폴더 안에 파일이름 읽기

jpg 파일 리스트 저장

```
import glob
```

```
train_cat_fnames = glob.glob(train_cats_dir+'/*.jpg')
```

```
train_dog_fnames = glob.glob(train_dogs_dir+'/*.jpg')
```

이미지 갯수 세기

```
len(train_cat_fnames), len(train_dog_fnames)
```

```
(1000, 1000)
```

폴더 안에 이미지 열기

이미지 한개씩 열 때

```
from PIL import Image
```

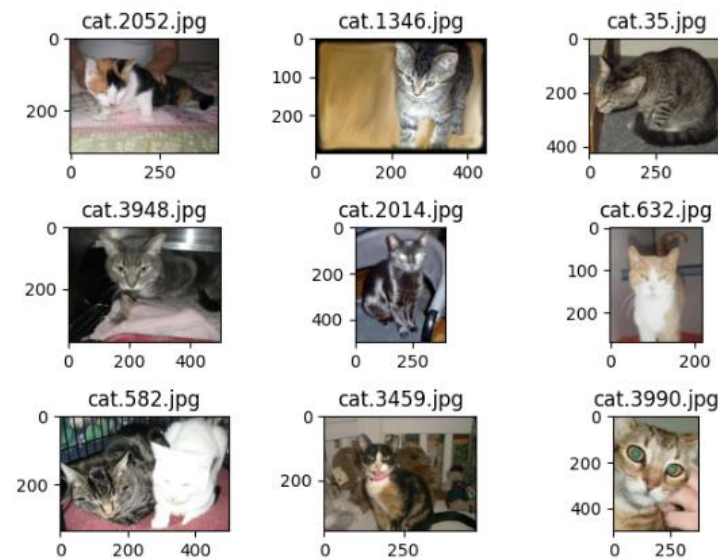
```
Image.open(train_cat_dir+'/cat.1.jpg').size
```

고양이 훈련데이터 시각화

고양이 훈련 데이터 9개 시각화

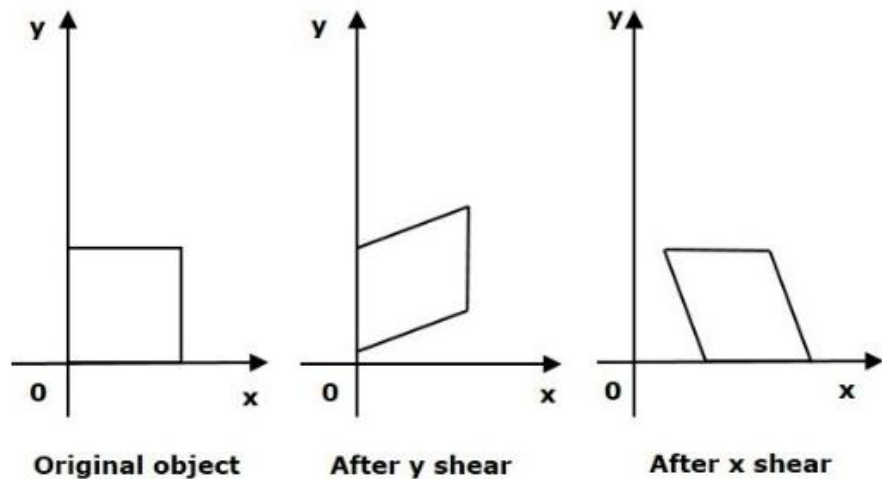
```
import matplotlib.pyplot as plt
from PIL import Image
```

```
for i in range(9):
    f = os.path.join(train_cat_dir,
os.listdir(train_cat_dir)[i])
    plt.subplot(3,3,i+1)
    plt.title(os.listdir(train_cat_dir)[i])
    plt.imshow(Image.open(f))
plt.tight_layout()
```



이미지 데이터 증강 설정

```
train_datagen = ImageDataGenerator(  
    rescale = 1.0/255., # 픽셀값 스케일 범위 [0,1]  
    rotation_range=40, # 40 도까지 임의 회전  
    width_shift_range=0.2, # 수평이동  
    height_shift_range=0.2, # 수직이동  
    shear_range=0.2, # 전단변환  
    zoom_range=0.2, # 20%까지 임의 확대  
    horizontal_flip=True) #수평플립  
test_datagen = ImageDataGenerator(  
    rescale = 1.0/255.) # 정규화
```



Flow from Directory

- 이미지 파일을 로딩하여 라벨을 가진 증강 이미지의 배치를 생성

```
train = train_datagen.flow_from_directory(  
    train_dir, #훈련이미지가 있는 폴더  
    target_size=(150,150), #이미지 사이즈 변경  
    class_mode='binary' #이진 레이블  
)  
  
test = test_datagen.flow_from_directory(  
    validation_dir, #검증이미지가 있는 폴더  
    target_size=(150,150), #이미지 사이즈 변경  
    class_mode='binary' #이진 레이블  
)
```

기본 CNN 모델

모델생성

```
model = Sequential()
model.add(Conv2D(32, 3, activation='relu', input_shape=(150,150,3)))
model.add(MaxPool2D(2,2))
model.add(Conv2D(64, 3, activation='relu'))
model.add(MaxPool2D(2,2))
model.add(Conv2D(128, 3, activation='relu'))
model.add(MaxPool2D(2,2))
model.add(Conv2D(128, 3, activation='relu'))
model.add(MaxPool2D(2,2))
model.add(Flatten())
model.add(Dropout(.5))
model.add(Dense(512, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.summary()
```

Model: "sequential"		
Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147584
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dropout (Dropout)	(None, 6272)	0
dense (Dense)	(None, 512)	3211776
dense_1 (Dense)	(None, 1)	513
=====		
Total params: 3453121 (13.17 MB)		
Trainable params: 3453121 (13.17 MB)		
Non-trainable params: 0 (0.00 Byte)		

컴파일 및 학습

컴파일

```
model.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=1e-4),  
metrics=['accuracy'])
```

체크 포인트, 조기 종료 설정

```
from keras.callbacks import ModelCheckpoint, EarlyStopping
```

```
save_file_name = 'cats_and_dogs_cnn_model.h5'
```

```
checkpoint = ModelCheckpoint(save_file_name, monitor='val_accuracy',  
                             verbose=1, save_best_only=True, mode='auto')
```

```
earlystopping = EarlyStopping(monitor='val_accuracy', patience=5)
```

모델 학습

```
history = model.fit(train_flow, epochs=30, validation_data=test_flow, verbose=2,  
callbacks=[checkpoint, earlystopping])
```

히스토리 그래프

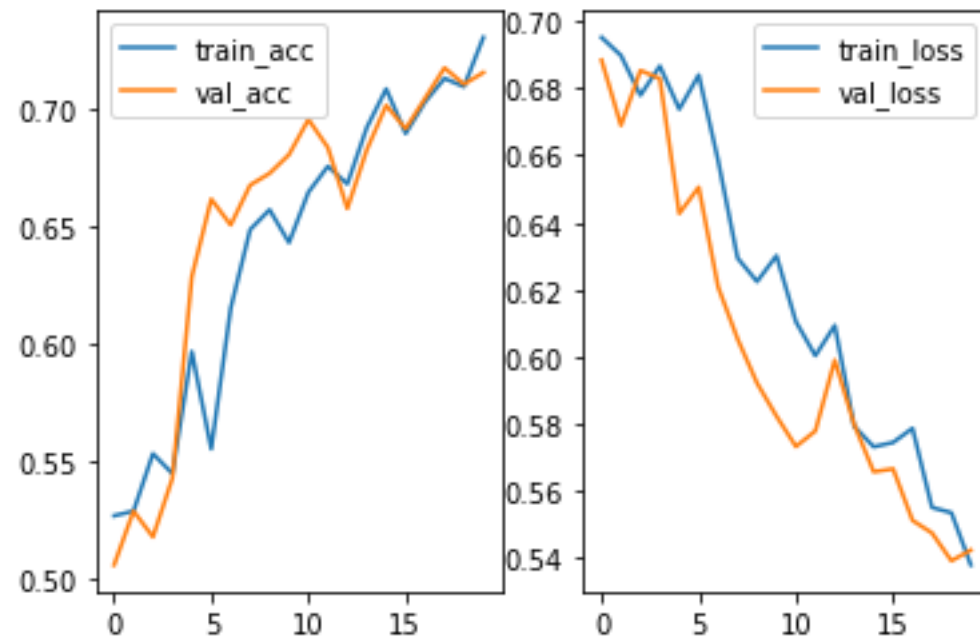
```
import matplotlib.pyplot as plt

plt.figure(figsize=(15,5))

plt.subplot(121)
plt.title('Loss')
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='test')
plt.legend()

plt.subplot(122)
plt.title('Accuracy')
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='test')
plt.legend()

plt.show()
```



테스트 이미지 분류

```
# 테스트 이미지 분류
```

```
import numpy as np
```

```
from keras.preprocessing import image
```

```
path = '/content/chihuahua.jpg'
```

```
img=image.load_img(path, target_size=(150, 150))
```

```
img.size # (150, 150)
```

```
x=image.img_to_array(img)
```

```
x.shape # (150, 150, 3)
```

```
x=np.expand_dims(x, axis=0)
```

```
x.shape # (1, 150, 150, 3)
```

```
classes = model.predict(x)
```

```
print(classes[0]) #[1.]
```

```
if classes[0]>0:
```

```
    print(path + " is a dog")
```

```
else:
```

```
    print(path + " is a cat")
```

```
    /content/chihuahua.jpg is a dog
```



```
array([[227., 227., 229.],
       [229., 229., 231.],
       [227., 227., 229.],
       ...,
       [ 36.,  35.,  31.],
       [ 39.,  38.,  34.],
       [ 37.,  33.,  30.]])
```

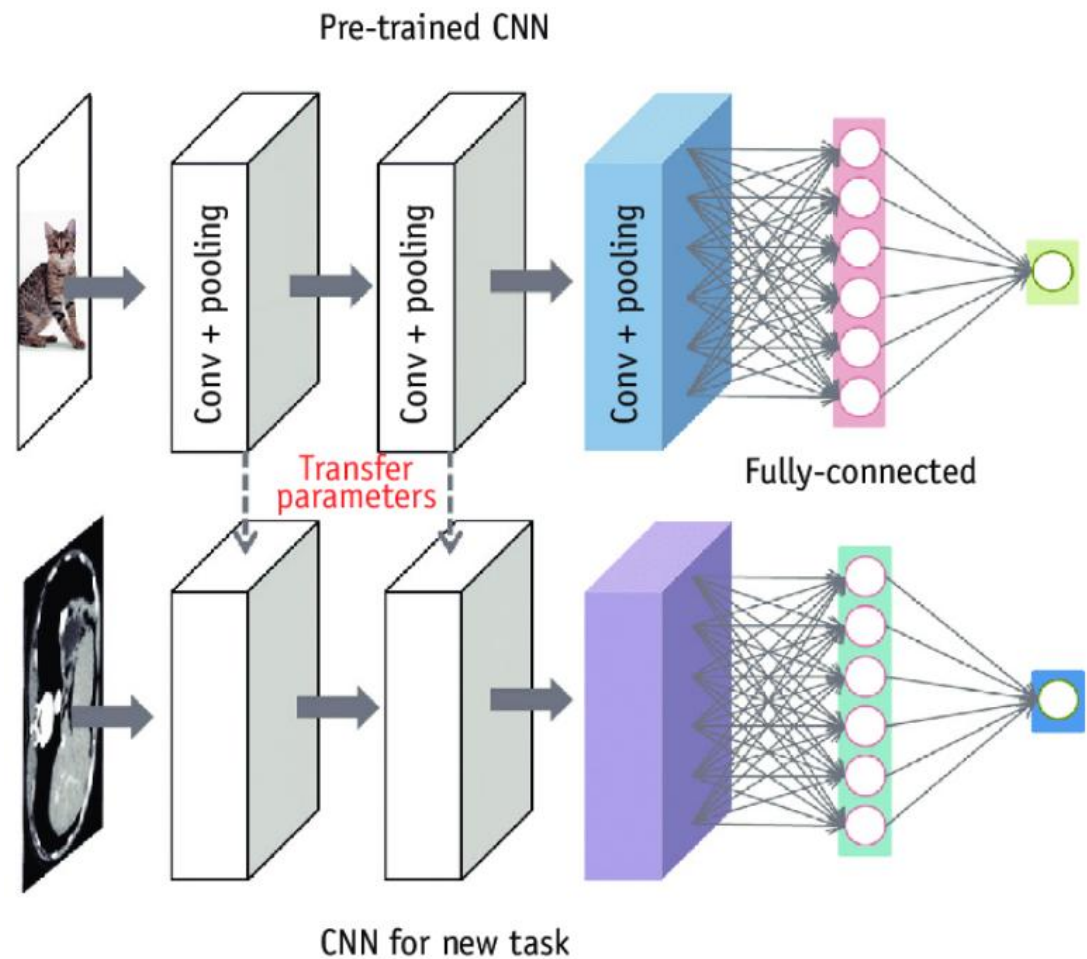
```
[[227., 227., 229.],
 [222., 222., 222.],
 [220., 220., 220.],
```

```
array([[[[227., 227., 229.],
         [229., 229., 231.],
         [227., 227., 229.],
         ...,
         [ 36.,  35.,  31.],
         [ 39.,  38.,  34.],
         [ 37.,  33.,  30.]])])
```

```
[[227., 227., 229.],
 [222., 222., 222.],
 [220., 220., 220.],
```

Transfer Learning 전이학습

- Process of taking pretrained model (usually trained on large dataset, such as ImageNet) and “fine-tuning” model with new dataset. 사전 훈련된 모델 (일반적으로 ImageNet과 같은 대규모 데이터 세트로 미리 훈련)을 가지고 새로운 데이터 세트에 “미세 조정”을 하는 프로세스



Pre-Trained Model 사전학습 모델

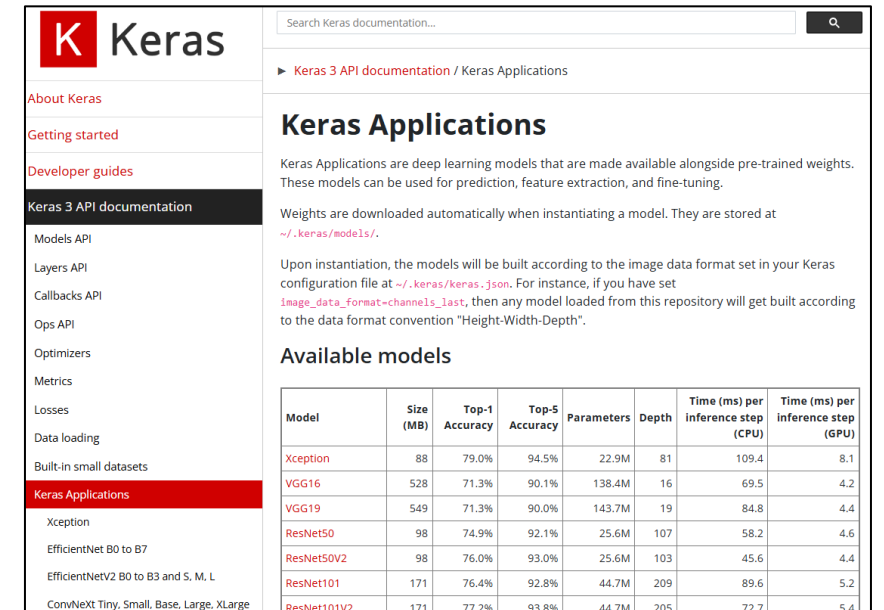
- A saved network that was previously trained on a large dataset, typically on a large-scale image-classification task. You either use the pretrained model as is or use transfer learning to customize this model to a given task. 대규모 이미지 분류 작업에 대해 미리 훈련되고 저장된 네트워크. 사전 학습된 모델을 그대로 사용하거나 전이 학습을 사용하여 이 모델을 지정된 작업에 맞게 바꿀 수 있음
- Used for new and similar problem. 새로운 문제나 비슷한 문제에 사용

CNN Architectures 아키텍처 구조

- Popular CNN architectures won in ILSVRC (ImageNet Large Scale Vision Recognition Challenge) competitions. ILSVRC 대회에서 우승했던 인기 있는 CNN 아키텍처
 - LeNet-5
 - AlexNet
 - VGGNet
 - GoogLeNet
 - ResNet
 - Inception
- Participants are provided with 1.4 millions of images. 참가자에게는 140만 개의 이미지가 제공

Pre-Trained Models in Keras

- Keras Applications
 - Xception
 - EfficientNet B0 to B7
 - VGG16 and VGG19
 - ResNet and ResNetV2
 - MobileNet and MobileNetV2
 - DenseNet
 - NasNetLarge and NasNetMobile
 - InceptionV3
 - InceptionResNetV2



The screenshot shows the Keras Applications documentation page. The left sidebar contains a navigation menu with links to 'About Keras', 'Getting started', 'Developer guides', 'Keras 3 API documentation', 'Models API', 'Layers API', 'Callbacks API', 'Ops API', 'Optimizers', 'Metrics', 'Losses', 'Data loading', 'Built-in small datasets', 'Keras Applications', 'Xception', 'EfficientNet B0 to B7', 'EfficientNetV2 B0 to B3 and S, M, L', and 'ConvNext Tiny, Small, Base, Large, XLarge'. The main content area is titled 'Keras Applications' and includes a search bar, a breadcrumb 'Keras 3 API documentation / Keras Applications', and a description of Keras Applications. Below the description is a table titled 'Available models' with columns for Model, Size (MB), Top-1 Accuracy, Top-5 Accuracy, Parameters, Depth, Time (ms) per inference step (CPU), and Time (ms) per inference step (GPU). The table lists models such as Xception, VGG16, VGG19, ResNet50, ResNet50V2, ResNet101, and ResNet101V2.

Keras Applications

Keras Applications are deep learning models that are made available alongside pre-trained weights. These models can be used for prediction, feature extraction, and fine-tuning.

Weights are downloaded automatically when instantiating a model. They are stored at `~/.keras/models/`.

Upon instantiation, the models will be built according to the image data format set in your Keras configuration file at `~/.keras/keras.json`. For instance, if you have set `image_data_format=channels_last`, then any model loaded from this repository will get built according to the data format convention "Height-Width-Depth".

Available models

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4

<https://keras.io/api/applications/>

Lab #4

- Complete the tutorial on <https://keras.io/api/applications/>
 - Classify ImageNet classes with ResNet50

```
import keras
from keras.applications.resnet50 import ResNet50
from keras.applications.resnet50 import preprocess_input,
import numpy as np

model = ResNet50(weights='imagenet')

img_path = 'elephant.jpg'
img = keras.utils.load_img(img_path, target_size=(224, 224))
x = keras.utils.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

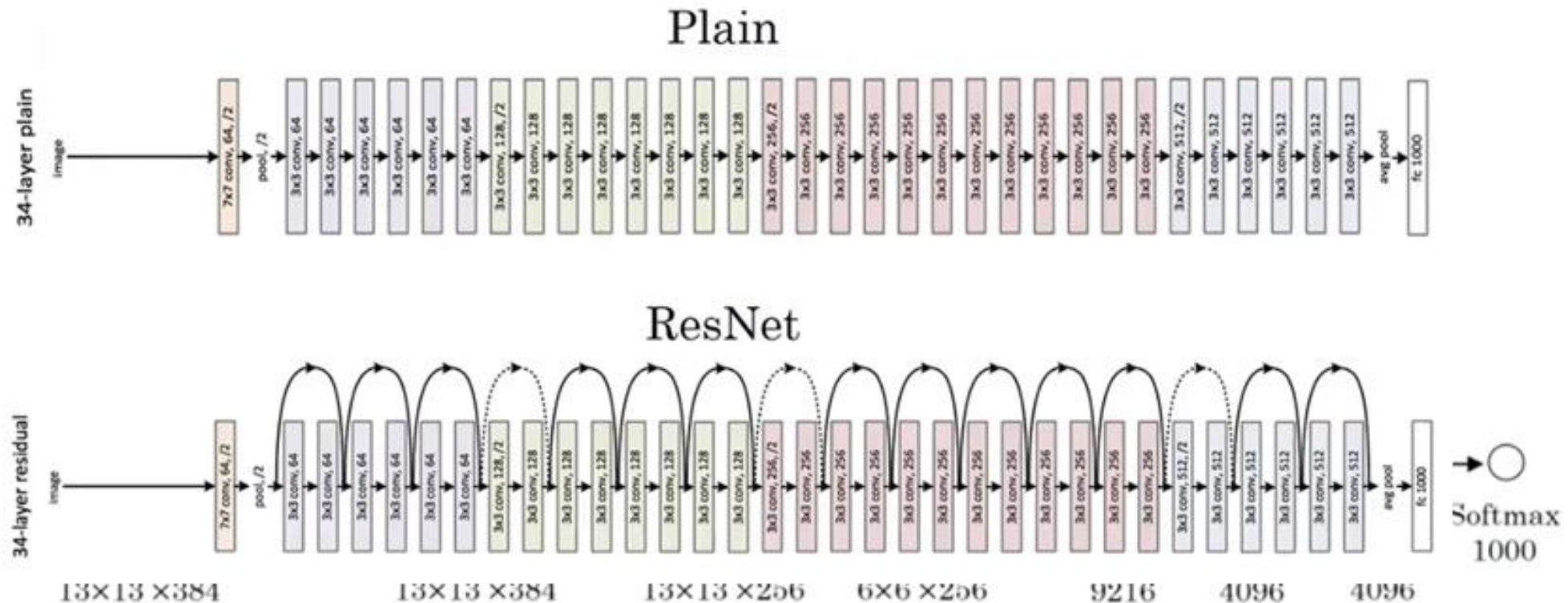
preds = model.predict(x)
# decode the results into a list of tuples (class, description)
# (one such list for each sample in the batch)
print('Predicted:', decode_predictions(preds, top=3)[0])
# Predicted: [(u'n02504013', u'Indian_elephant', 0.826582),
```

ResNet (Residual Neural Network)

- Introduced a concept called “skip connections.”
“연결 건너뛰기”라는 개념을 도입

Error - 3.57

224x224 pixels colored images



ResNet 모델 구축

```
# 필요한 라이브러리 import
import keras

from keras.applications.resnet50 import ResNet50 # ResNet50 모델
from keras.applications.resnet50 import preprocess_input,
decode_predictions # 전처리 및 예측 결과 디코딩
import numpy as np # 수치 계산을 위한 라이브러리

# ImageNet 데이터셋으로 사전 학습된 ResNet50 모델 불러오기
model = ResNet50(weights='imagenet')
```

이미지 전처리

예측에 사용할 이미지 파일 경로 지정

```
img_path = 'elephant.jpg'
```

이미지를 불러오고 ResNet50 입력 크기인 (224, 224)로 리사이즈

```
img = keras.utils.load_img(img_path, target_size=(224, 224))
```

이미지를 NumPy 배열로 변환

```
x = keras.utils.img_to_array(img)
```

배열의 차원을 (1, 224, 224, 3)으로 확장 → 배치 크기 포함

```
x = np.expand_dims(x, axis=0)
```

ResNet50에 맞는 전처리 수행 (픽셀 정규화 등)

```
x = preprocess_input(x)
```

테스트 데이터로 예측

모델에 입력 데이터를 넣고 예측 수행

```
preds = model.predict(x)
```

예측 결과를 해석 가능한 형태로 디코딩 (클래스 ID, 설명, 확률)

top=3 → 확률이 높은 상위 3개의 클래스만 추출

```
print('Predicted:', decode_predictions(preds, top=3)[0])
```

예시 출력:

```
# Predicted: [('n02504013', 'Indian_elephant', 0.82658225),
```

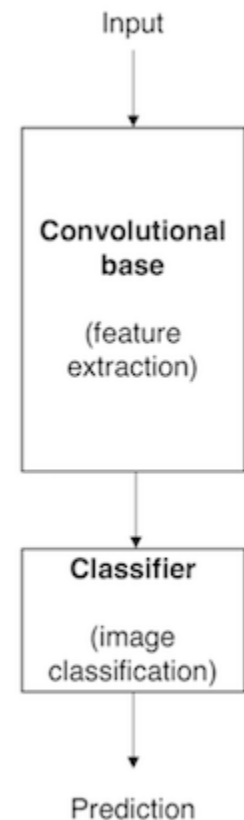
```
#           ('n01871265', 'tusker', 0.1122357),
```

```
#           ('n02504458', 'African_elephant', 0.061040461)]
```

Feature Extractor and Classifier

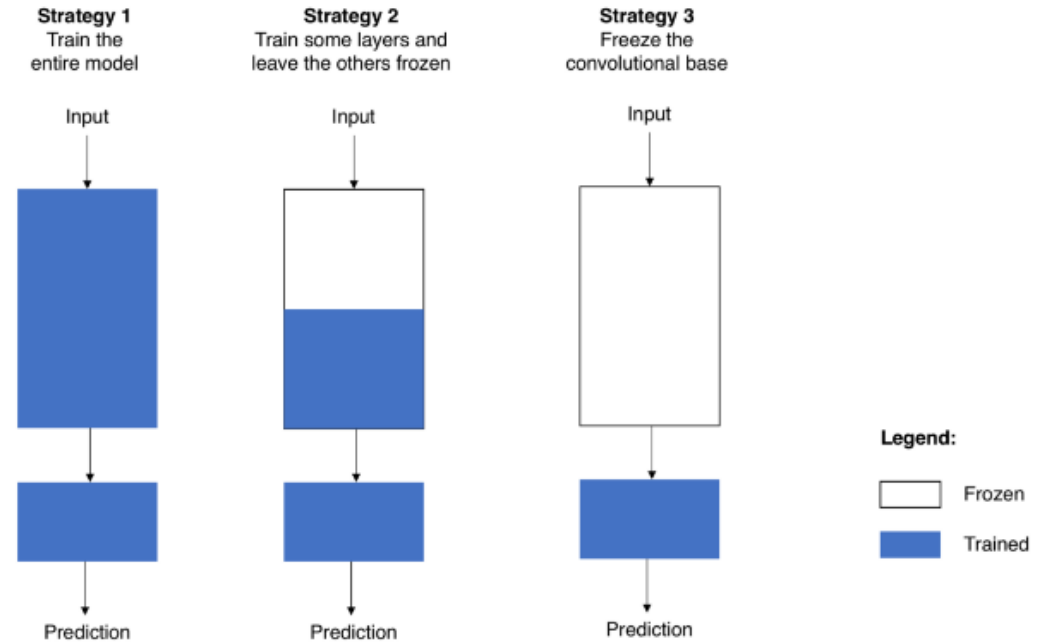
특성추출기와 분류기

- Feature Extractor 특성 추출기
 - 컨볼루션 층과 풀링 층으로 구성되며 ImageNet 데이터로 이미 학습되어 있음
 - 출력데이터: bottleneck 또는 feature vector
- Classifier 분류기
 - Fully connected layer (완전 연결층)으로 구성되며 추출된 특성을 입력 받아 분류를 진행
 - 오버피팅을 줄이기 위해 Dropout, BatchNormalization 레이어 등을 추가



Fine-Tuning 파인 튜닝

- 사전학습 모델의 가중치를 미세하게 조정하는 방법
- 분류하는 데이터의 종류와 갯수에 따라 사전학습 모델의 가중치 일부를 재학습 시키거나 모든 가중치를 처음부터 다시 학습 시킬 수 있음



전체를 다 훈련할 수 있게 함

```
base_model.trainable = True
```

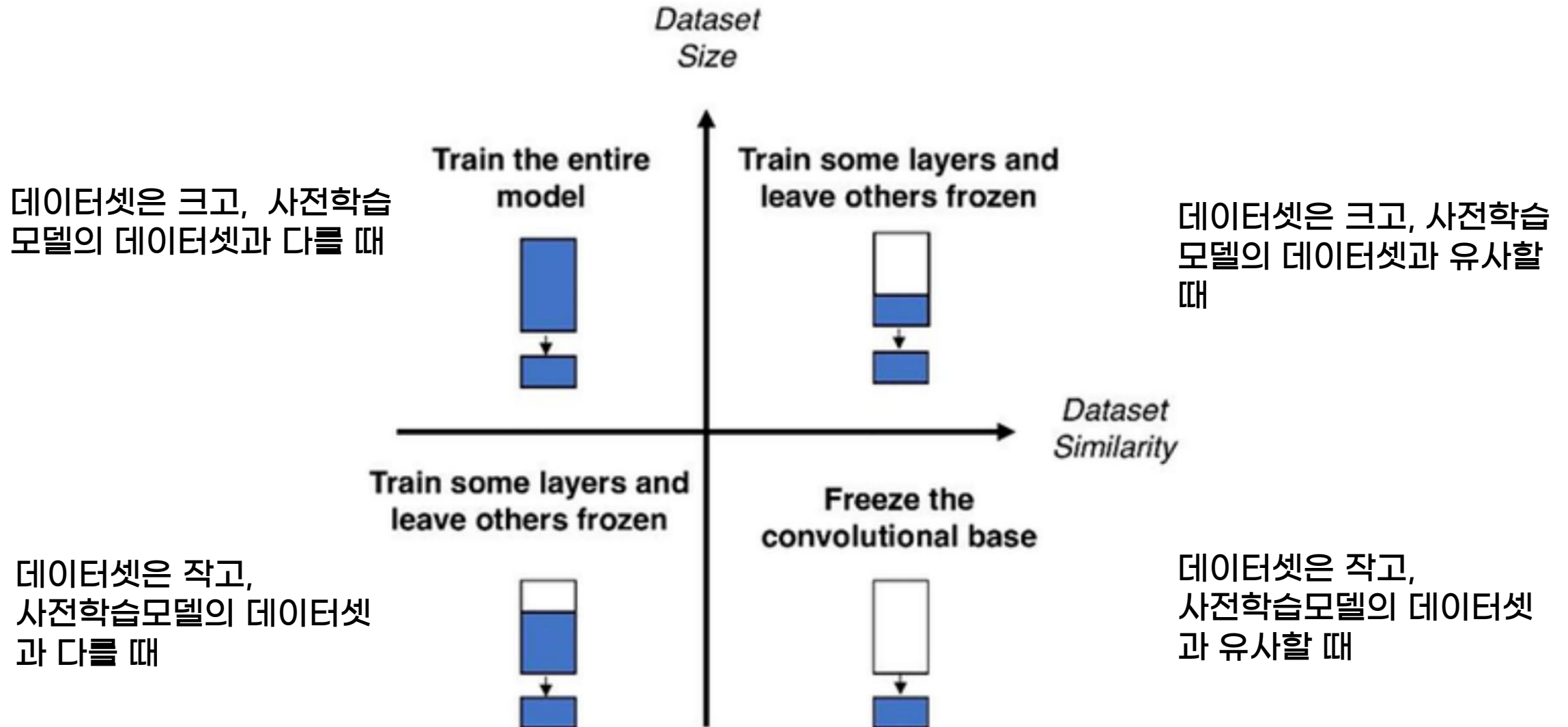
일부 특성추출기를 동결

```
for layer in base_model.layers[143:]:  
    layer.trainable = True
```

특성추출기를 동결

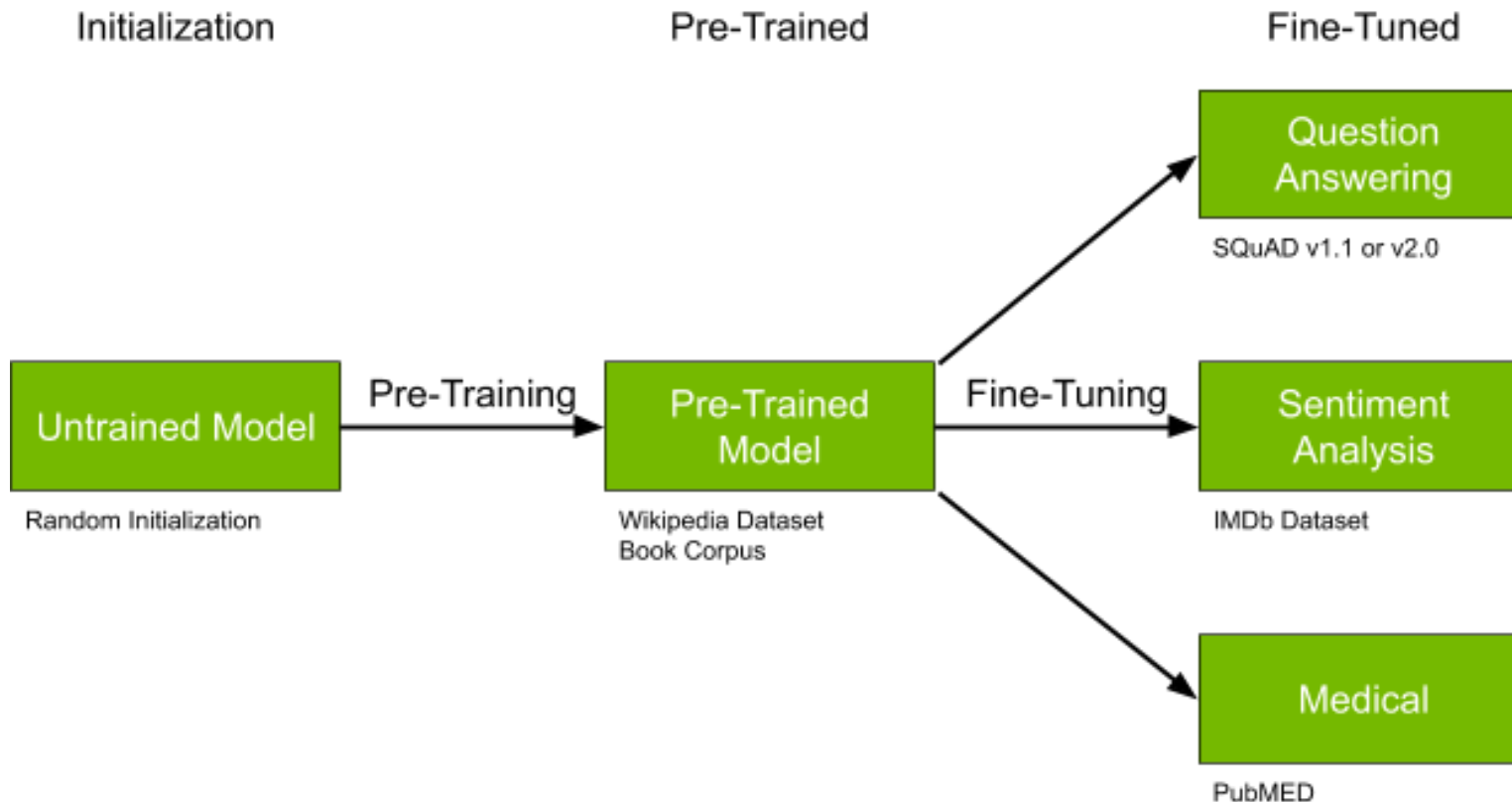
```
base_model.trainable = False
```

데이터 크기와 유사도에 따른 전이학습의 사용



Fine-Tuning Example

- BERT (Bidirectional Encoder Representations from Transformers)의 핵심 모델은 대규모의 일반 데이터 세트에서 사전 훈련한 다음 질문/답변, 감정 분석 또는 명명된 엔터티 인식과 같은 다양한 작업을 수행하도록 미세 조정할 수 있음



Typical Fine-Tuning Workflow

1. 기본 모델을 개체화하고 사전 훈련된 가중치를 로드
2. `trainable = False`를 설정하여 기본 모델의 모든 레이어를 동결
3. 기본 모델의 하나 또는 여러 레이어의 출력 위에 새 모델을 생성
4. 새로운 데이터 세트에 대한 새로운 모델을 훈련
5. 모델이 새 데이터에 수렴되면 기본 모델의 전체 또는 일부를 해제하고 매우 낮은 학습률로 전체 모델을 다시 훈련할 수 있음

특성 추출기만 가져오기

- Instantiate a base model and load pre-trained weights into it. 기본 모델을 개체화하고 사전 훈련된 가중치를 로드

```
from tensorflow.keras.applications import *  
  
base_model = VGG16(weights='imagenet',  
                    input_shape=(240, 240, 3),  
                    include_top=False) # 특성추출기만 사용  
base_model.summary()
```

특성 추출기, 분류기 가져오기

```
from tensorflow.keras.applications.inception_v3 import InceptionV3
base_model = InceptionV3(input_shape=(150, 150, 3),
                          include_top=True, #사전학습 모델의 특성추출기와
                                             분류기 둘다 가져옴
                          weights=None) #적용할 가중치 지정
base_model.summary()
```

Weights

- None: 임의로 초기화 (random initialization)된 가중치를 적용
- imagenet: ImageNet에 대해 사전 훈련된 가중치를 적용 (Default 값)

Base Model 구조 확인

```
from tensorflow.keras.applications import *
```

```
base_model = VGG16(weights='imagenet',  
                    input_shape=(240, 240, 3),  
                    include_top=False) # 특성추출기만 가져옴
```

```
base_model.input # 입력 텐서
```

```
<KerasTensor: shape=(None, 240, 240, 3) dtype=float32 (created by layer 'input_1')>
```

```
base_model.output # 출력 텐서
```

```
<KerasTensor: shape=(None, 7, 7, 512) dtype=float32 (created by layer 'block5_pool')>
```

```
print(base_model.layers[-1]) # 마지막 레이어
```

```
<keras.src.layers.pooling.max_pooling2d.MaxPooling2D at 0x7802582870a0>
```

```
print(base_model.layers.pop()) #마지막 레이어 제거
```

```
<keras.src.layers.pooling.max_pooling2d.MaxPooling2D at 0x7802582870a0>
```

전이학습 모델 위에 새로운 분류기를 추가

- Create a new model on top of the output of one (or several) layers from the base model. 기본 모델의 하나 또는 여러 레이어의 출력 위에 새 모델을 생성

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPool2D, Dense, Flatten, Dropout
from keras.optimizers import Adam

model = Sequential()
model.add(base_model)
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=1e-4), metrics=['accuracy'])
model.summary()
```

Fine-tuning시 학습
률을 낮게 설정해
pre-trained
weights를 조금씩
업데이트 해줌

모델 컴파일

- Train your new model on your new dataset. 새로운 데이터 세트에 대한 새로운 모델을 훈련

모델 컴파일

```
model.compile(optimizer = 'adam',  
              loss = 'sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

가중치의 훈련가능 여부

- 훈련하기 전 base model 의 가중치가 바뀌는 것을 막기 위해 이를 동결

- 베이스 모델 전체를 동결하는 경우

```
base_model.trainable=False
```

- 각 레이어를 순차적으로 동결하는 경우

```
for layer in base_model.layers:  
    layer.trainable = False
```

모든 레이어 동결

- Freeze all layers in the base model by setting trainable = False.
훈련 가능 = False를 설정하여 기본 모델의 모든 레이어를 동결

```
# 전체 모델의 가중치 동결
```

```
base_model.trainable = False
```

```
# 훈련 가능한 가중치의 갯수 확인
```

```
len(model.trainable_weights) #4
```

동결 해제

- Once your model has converged on the new data, you can try to unfreeze all or part of the base model and retrain the whole model end-to-end with a very low learning rate. 모델이 새 데이터에 수렴되면 기본 모델의 전체 또는 일부를 해제하고 매우 낮은 학습률로 전체 모델을 다시 훈련할 수 있음

모든 층의 동결 해제

```
base_model.trainable = True
```

훈련할 수 있는 가중치 갯수

```
len(model.trainable_weights) #30
```

일부 레이어 해제

```
for layer in base_model.layers[143:]:  
    layer.trainable = True
```

가중치 저장 및 조기종료 설정

```
from keras.callbacks import ModelCheckpoint, EarlyStopping

save_file_name = 'cifar10_transfer_model.h5'
checkpoint = ModelCheckpoint(save_file_name, monitor='val_loss',
                             verbose=1, save_best_only=True,
                             mode='auto')
earlystopping = EarlyStopping(monitor='val_loss', patience=5)
```

모델 훈련

- Train your new model on your new dataset. 새로운 데이터 세트에 대한 새로운 모델을 훈련

```
# 모델 훈련
```

```
history = model.fit(x_train, y_train, epochs = 100, validation_data=(x_test, y_test), batch_size= 256, callbacks=[checkpoint, earlystopping])  
model.save("cifar10_resnet_model.h5")
```

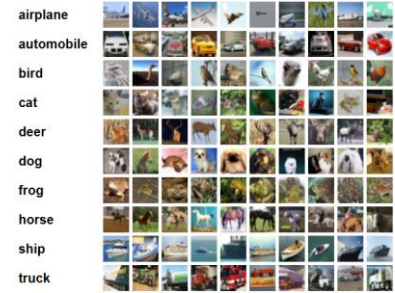
전이학습 모델 튜닝기법

- 옵티마이저 변경
- BatchNormalization 추가
- Dropout 추가
- 사전학습 모델 전체 학습
 - `base_model.trainable = True`
- 데이터 전처리 및 증강
- 분류기에 은닉층 및 노드 추가
- Learning decay를 이용한 유동적인 학습 진행

Learning decay:

- 학습률(learning rate)을 점진적으로 감소
- 초반엔 빠르게 학습. 후반엔 작게 조정하여 안정적으로 수렴

Lab #4



- CIFAR-10 데이터를 이용하여 전이학습을 수행하시오
- ResNet50의 특성추출기와 분류기를 그대로 이용하여 데이터를 예측하시오 (컴파일 필요, 필요한 경우 이미지 차원 및 사이즈 변경)
- ResNet50의 특성추출기만 가져와서 모델을 만든 후 다음 학습을 수행하시오
 - 특성추출기 동결 후 학습
 - 특성추출기 143층부터 해제한 후 학습 (0.001 학습률)
- ResNet50의 특성추출기만 가져와서 모델을 만든 후 데이터를 증강한 후 다음 학습을 수행하시오
 - 특성추출기 동결 후 학습

사전학습 모델 가져오기 (ResNet50)

- ResNet50의 특성추출기와 분류기를 그대로 이용

```
from tensorflow.keras.applications.resnet50 import  
ResNet50
```

베이스 모델 가져오기

```
base_model = ResNet50(input_shape=(224, 224, 3),  
weights='imagenet', include_top=True) # 특성추출기, 분류기 다  
가져옴
```

- ResNet50의 특성추출기만 가져오기

베이스 모델 가져오기

```
base_model = ResNet50(input_shape = (32, 32, 3), weights =  
'imagenet', include_top=False) # 특성추출기만 가져옴
```

사전학습 모델 가져오기 (기타)

```
from tensorflow.keras.applications import *
```

```
mobilenet = MobileNet(weights=None, input_shape=None,  
include_top=True)
```

```
efficientnet = EfficientNetB0(weights=None, input_shape=None,  
include_top=True)
```

```
resnet50 = ResNet50(weights=None, input_shape=None,  
include_top=True)
```

```
xception = Xception(weights=None, input_shape=None,  
include_top=True)
```

이미지 차원 및 사이즈 변경

```
from tensorflow.keras.applications.resnet50 import preprocess_input

inputs = tf.expand_dims(x_test[0], axis=0) #차원추가 (None, 224, 224, 3)
x = tf.keras.layers.experimental.preprocessing.Resizing(224,
224)(inputs) #사이즈 변경
x = preprocess_input(x) #정규화
```

예측 및 시각화

```
from tensorflow.keras.applications.resnet50 import  
decode_predictions
```

예측

```
preds = base_model.predict(x)
```

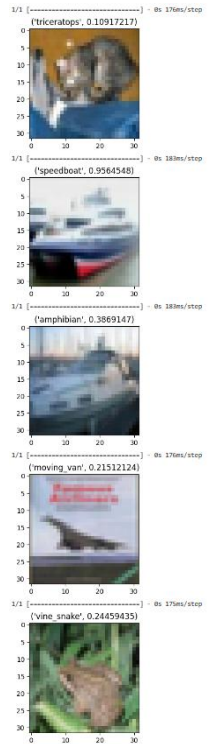
시각화

```
fig= plt.figure(figsize=(3,3))
```

```
plt.title(decode_predictions(preds, top=1)[0][0][1:3]) # 가장 확률이  
높은 예측 이름 추출해서 타이틀로 넣음
```

```
plt.imshow(x_test[0])
```

```
plt.show()
```



전이학습 모델 + 전처리

사이즈를 바꿔서 모델 생성

```
import tensorflow as tf

from keras import Input, Model

from keras.layers import Conv2D, MaxPooling2D, Dense, Flatten, Dropout


inputs = Input(shape=(32, 32, 3))

x = tf.keras.layers.experimental.preprocessing.Resizing(224, 224)(inputs)

x = tf.keras.applications.resnet50.preprocess_input(inputs)

x = base_model(x, training = False)

x = Flatten()(x)

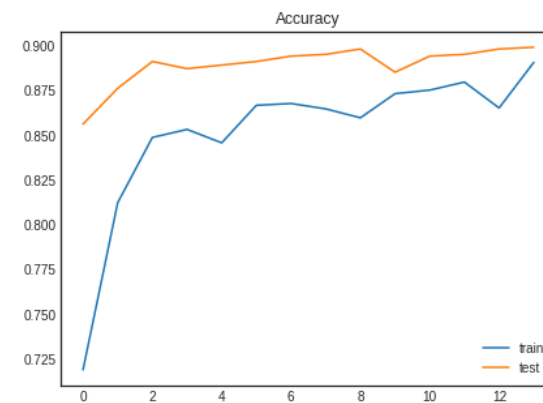
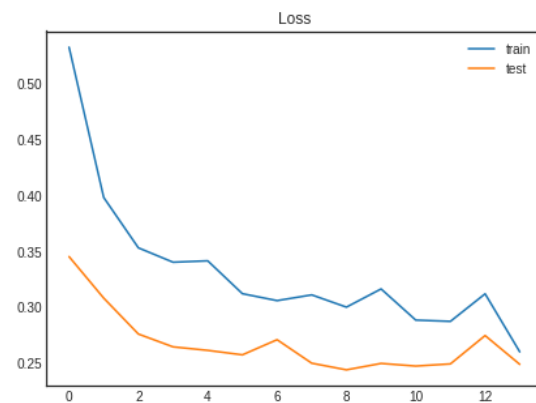
outputs = Dense(10, activation = 'softmax')(x)

model = Model(inputs, outputs)

model.summary()
```

히스토리 그래프

```
plt.figure(figsize=(15,5))
plt.subplot(121)
plt.title('Loss')
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='test')
plt.legend()
plt.subplot(122)
plt.title('Accuracy')
plt.plot(history.history['acc'], label='train')
plt.plot(history.history['val_acc'], label='test')
plt.legend()
plt.show()
```



학습률 조정

```
# 모델 컴파일 (학습률 0.001)
```

```
from tensorflow.keras.optimizers import Adam
```

```
model.compile(optimizer = Adam(learning_rate= 0.001),  
              loss = 'sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

ChatGPT로 성능 개선 전략 탐색

- 모델 구조 개선 프롬프트 예

"ResNet50 기반 CNN 모델의 구조를 개선하고 싶은데, 어떤 레이어를 추가하면 좋을까요?"

- 하이퍼파라미터 튜닝 프롬프트 예

"Adam 옵티마이저를 사용할 때, 학습률을 어떻게 설정하는 게 좋을까요?"

- 전이학습 세부 설정 프롬프트 예

"ResNet50에서 어느 layer부터 훈련을 다시 시작하는 것이 좋을까요?"

- 데이터 증강 전략 프롬프트 예

"과적합이 있는 모델에 효과적인 이미지 증강 전략을 추천해줘."